Quality validation

Quality validation I

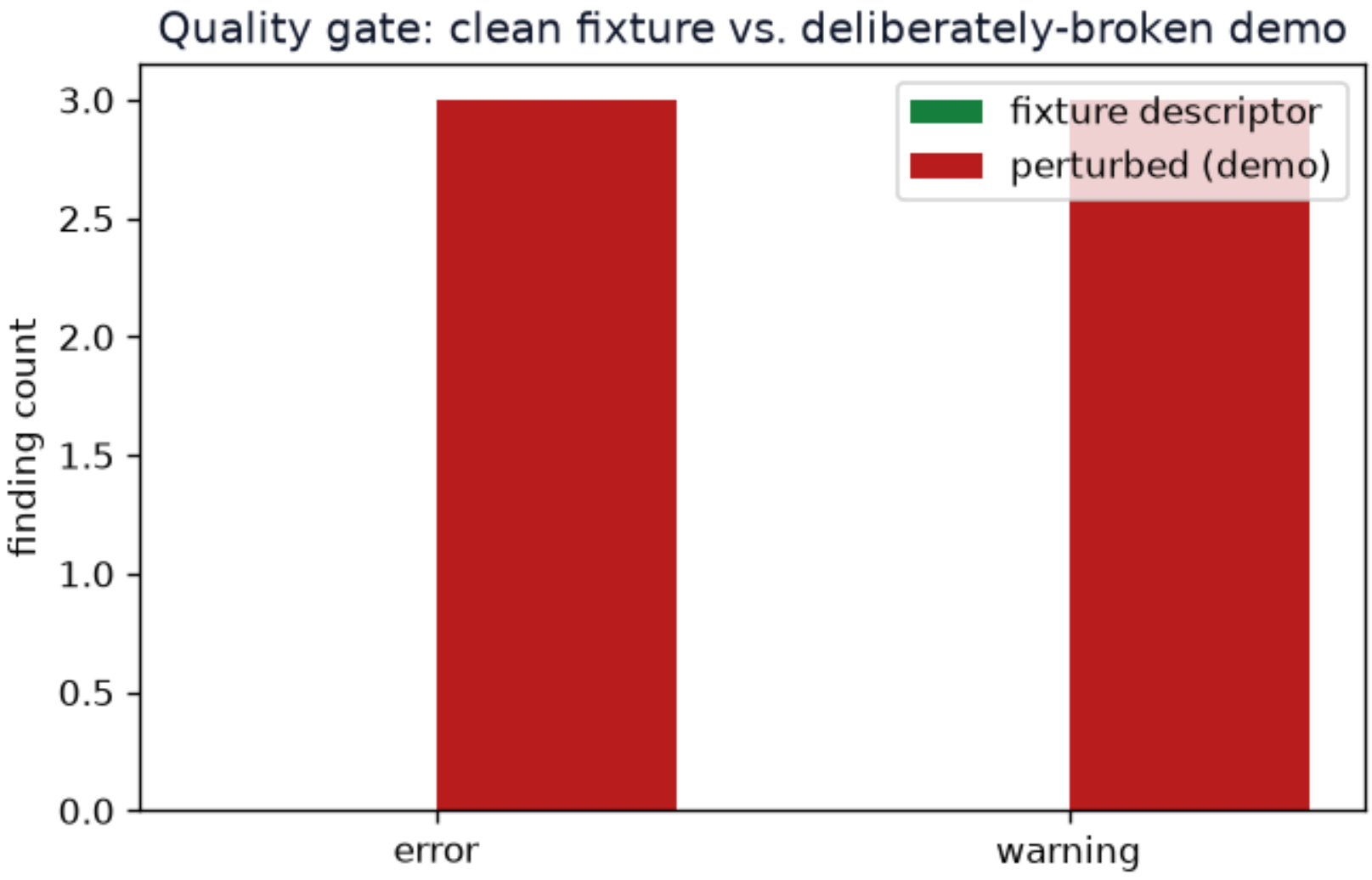
The quality gate is what separates a descriptor that merely *claims* to describe a dataset from one that *provably* does. It has two layers: structural validation of the descriptor, and byte-level reconciliation of the descriptor against the files it names.

The validation gate reacts to defects I

`validate_descriptor()` emits severity-tagged findings for missing required keys, unsafe or duplicate file paths, malformed checksums, unknown media types, duplicate or malformed fields, constraint violations, missing primary keys, and thin provenance. `build_descriptor_report()` folds these into a readiness score, penalising each error and warning.

To show the gate is not vacuous, fig. 4 compares the shipped fixture against a deliberately-perturbed copy produced by `demo_broken_descriptor()` (which strips the license, corrupts a checksum, zeroes a row count, and removes a unit). The clean fixture produces zero findings; the perturbed demo produces several errors and warnings.

The validation gate reacts to defects I



The validation gate reacts to defects I

Figure 4: Quality gate: validation findings by severity for the clean fixture descriptor (left, zero findings) versus a deliberately-broken demonstration copy (right). The demonstration perturbation is clearly named in code and never treated as real data — it exists only to prove the gate reacts. Counts come from `severity_counts()`.

Descriptor↔file verification I

A checksum is only meaningful if something checks it. `verify_descriptor_files()` recomputes the sha256 digest and data-row count of every declared file that is present on disk and compares them to the descriptor's declarations, reporting each file as `verified`, `checksum_mismatch`, `row_mismatch`, or `absent`. Files a descriptor references but does not bundle are reported as `absent` rather than as failures — verification is asserted only for bytes that are actually present, which is what makes the metadata-only release boundary honest.

Descriptor↔file verification I

fig. 5 shows this reconciliation for the shipped fixture: both files verify, declared and actual row counts agree, and each recomputed checksum matches its declaration, so the readiness score is unpenalised.

Descriptor↔file verification I

Descriptor↔file verification (readiness 1.0)

file	declared rows	actual rows	checksum	status
xtures/measurements.	12	12	match	verified
fixtures/subjects.csv	6	6	match	verified

Figure 5: Descriptor↔file verification: declared versus recomputed row counts and checksum agreement per file, from `verify_descriptor_files()`. Both shipped fixture files verify against their bytes on disk; the title reports the descriptor's readiness score. If a fixture file were edited without updating the descriptor, the corresponding row would flip to a mismatch status and the test suite would fail.

Machine-readable release manifest I

`build_release_bundle_manifest()` packages the above into a deterministic, JSON-ready manifest containing the schema fingerprint, per-file checksums and row counts, field-level unit/bounds/enumeration summaries, provenance steps, and the readiness verdict — carrying metadata and checksums, never the dataset bytes. The thin script `scripts/generate_release_artifacts.py` writes this manifest (plus the readiness report and constraint summary) under `output/reports/` for pre-publication review.

Validation evidence I

Every capability above is exercised by the zero-mock `tests/` suite: structural validation and negative controls, order-independent schema fingerprints, byte-level verification against real temporary files, figure-data preparation, and an integration test that runs the figure script end to end and asserts real PNGs are written. Coverage exceeds the 90% project gate with no mocks.